

A I P O C K E T A G E N C Y

aipocketagency.com

[apa · kit 04]

Discovery → MVP Prompt Pack

The sequenced prompts I use to turn a 30-minute discovery call into shipped software in days, not weeks. Eight prompts. One real case study. The same machine that closed Patrick.

KIT

04 / 05

PRICE

\$15 USD

FORMAT

11 pages

TABLE OF CONTENTS

01	Why the gap between call and code is where deals die	p.2
02	The discovery-call framework (nine questions)	p.3
03	Prompt 1 — Transcript-to-Spec	p.5
04	Prompt 2 — Spec-to-Feature-List	p.6
05	Prompt 3 — Feature-to-Build-Prompts sequence	p.7
06	Prompt 4 — Branding extraction	p.8
07	Prompt 5 — Migration path	p.9
08	Prompt 6 — Handoff doc	p.10
09	Case study — Patrick, end to end	p.11

SECTION 1.

Why the gap between call and code is where deals die

The deal is signed. The discovery call went well. You have 45 minutes of rambling audio and an inbox full of “looking forward to seeing what you build.” Two weeks later you’re still rewriting the spec. The client is starting to wonder.

I have run this exact sequence on every paid build I’ve shipped. The first time I did it without the sequence — a contractor SaaS for a friend in 2024 — it took me six weeks to ship something I could have shipped in five days. The bottleneck wasn’t the code. It was the translation: rough call notes → spec → feature list → build prompts → working product. Every step lost fidelity. Every revision started a new argument about what the client had actually asked for.

This kit is the sequence I now run on every build. Eight prompts in order. Each one consumes the output of the prompt before it. The discovery call goes in at the top; a deployed MVP comes out the bottom. The case study at the end is Patrick — Fresh Page Home Improvement, a real contractor SaaS shipped in four days, billed at \$3,500 plus monthly hosting.

One closed build pays this kit back 233x. That math is the only reason to buy a prompt pack — and it’s the math I’m willing to put my receipts behind.

WHO THIS IS FOR

Operators who already run discovery calls — agency owners, freelance software builders, BOS-curious solo operators. If you haven’t sold a custom build yet, this kit isn’t your starting point. The Dispatch Playbook is. This kit assumes you have the sales motion and you’re losing time in the build motion.

SECTION 2.

The discovery-call framework (nine questions)

The call before the prompts. Get these nine answers in the first 30 minutes and the rest of the sequence flows. Miss one and you'll be on the phone again next week.

Discovery calls go sideways for one reason: the operator runs them like a sales call instead of a build call. Sales-call discovery is about closing. Build-call discovery is about **capturing enough fact to build**. The two need different question sets. The nine questions below are calibrated for the second. The opening and closing scripts wrap them.

The nine questions, in order:

1. **What does your business actually do?** Two sentences. Force them past the LinkedIn pitch. You're looking for the **operating verb** — "we replace roofs," "we run swim lessons," "we sell HVAC parts to contractors."
2. **Who pays you, and how much?** The customer side of the equation. Customer type, average transaction, transaction frequency. If they say "all kinds of customers," push back gently — pick the one that pays most.
3. **What's the most painful 30 minutes of your week?** Not the most painful task in general. The specific recurring 30 minutes that they dread. That's where the MVP lives.
4. **Walk me through what happens from [trigger] to [outcome].** Pick the operational verb from question one. Have them narrate the full flow. Take notes on every tool they mention, every handoff, every time they say "and then I have to remember to..."
5. **What tools are you using right now?** Names. CRMs, spreadsheets, Calendly, QuickBooks, paper invoices, whatever. The list of tools is the migration path's input — even if you decide not to migrate from any of them.
6. **What's the worst version of this going wrong?** The fear they're carrying. If their answer is small, the build is small. If their answer involves money, time, or a customer they can't lose, that's the wedge.
7. **Who else on your team will use this?** User count plus role. Plus the politics — "my husband will hate it" tells you something about the rollout. A solo operator with no one to please ships differently than a co-founder pair where one of the two needs to be sold separately.
8. **If we ship the smallest useful version in two weeks, what's the one feature that makes you say "this was worth it"?** The single MVP feature, named by them. You will build other features. But this one is the demo you give them on day one to make the rest of the build feel safe.

9. **What's the budget?** Don't skip this. If they can't answer it, you don't have a buyer yet — you have a curious tire-kicker. If they can, the number anchors everything below.

A short opening script ("Going to ask you nine things in 30 minutes — afterward I'll send a written spec and a build plan, no pressure to decide on this call") and a short close ("Two things I'll need before I draft the spec: your existing brand guide if you have one, and view-only access to your current tools so I can see real data") ship with the kit as ready-to-paste text.

RED FLAG

If the answer to question 6 is "I don't know" or the answer to question 9 is "however much it takes" — this is a rescue, not a build. Walk away. The kit ships a one-page "rescue vs build" decision tree for the moments when the call is the wrong call.

SECTION 3.

Prompt 1 — Transcript-to-Spec

The transcript goes in. A structured product spec comes out. The prompt is what makes the output usable instead of fluffy.

The Transcript-to-Spec prompt takes the raw call transcript plus one paragraph of project framing and produces a structured spec in five named sections: problem, audience, MVP feature set, post-MVP backlog, open questions. The shape of the output is the entire reason this prompt is in the kit — without the explicit output structure, the LLM produces a wall of summary you can't use.

You are a product spec writer. Read the discovery call transcript below.

Produce a structured product spec with EXACTLY these five sections:

1. PROBLEM — one paragraph naming the painful 30 minutes from the call.
2. AUDIENCE — two sentences: who pays, how much, how often.
3. MVP FEATURE SET — a numbered list of the smallest set of features that makes the customer say "this was worth it." Order by build dependency, not by what they mentioned first.
4. POST-MVP BACKLOG — a numbered list of features they want but don't need on day one. Don't argue with them — capture and park.
5. OPEN QUESTIONS — a numbered list of things they did not answer clearly enough to build from. Each one names what answer would resolve it.

Hard rules:

- Quote the customer directly when their words name the feature better than a generic description would.
- If a feature would require a third-party integration, name the integration (Stripe, Calendly, Twilio, QuickBooks, etc.) in the feature line.
- If the transcript contains a number (price, count, time), preserve it verbatim.
- Do not invent features the customer did not mention. If a feature is implied but not stated, put it in OPEN QUESTIONS instead.

Project framing: \[paste your one-paragraph framing\]

Transcript: \[paste the cleaned transcript\]

The prompt is annotated in the kit with notes on each rule and why it earns its place. The biggest one: **do not invent features the customer did not mention**. The default LLM behavior

is to fill gaps with plausible-sounding feature suggestions. Plausible-sounding features kill scope creep — you build them, the customer didn't ask for them, you get nothing for the work. The "put it in OPEN QUESTIONS instead" rule is the part that turns a guessing engine into a scoping engine.

SECTION 4.

Prompt 2 — Spec-to-Feature-List

The spec from Prompt 1 goes in. A ranked feature list with leverage scores comes out. The ranking is what tells you which lanes to spawn first.

Spec-to-Feature-List takes the MVP feature set from Prompt 1 and decomposes each feature into a shippable lane with leverage score, dependencies, and an estimate in “dispatch lanes” (one parallelizable unit of work). The leverage score is the part that matters — it’s the explicit “impact divided by buildability” math that tells you what to ship first.

You are a build planner. Read the product spec below.

For each feature in the MVP FEATURE SET, produce:

- Feature name (matching the spec)
- User story (one sentence in "As a X, I can Y, so that Z" form)
- Impact score (1-10) – how much does the customer care
- Buildability score (1-10) – how cheap is it to ship in this stack
- Leverage score = Impact / Buildability, rounded to 1 decimal
- Dependencies – other features that must ship first
- Dispatch lane count – how many parallelizable lanes this feature becomes (use 1 for tightly-coupled features, 2+ for fan-out features)

Hard rules:

- Order the output by leverage score, highest first.
- A feature with a dependency cannot have a higher rank than the feature it depends on, even if its leverage is higher.
- If two features have the same leverage score, order by buildability (cheap first).
- Buildability score reflects the actual project stack ([stack name]), not generic web app difficulty.

Project stack: [Next.js + Supabase + Stripe + Vercel, etc.]

Spec: [paste the spec from Prompt 1]

The output of this prompt is the build queue. Top three rows are what you ship in week one. The leverage-score math is what gives you a defensible “we’re not building feature X first” answer when the client wants their favorite-but-low-leverage idea promoted ahead of the schedule.

SECTION 5.

Prompt 3 — Feature-to-Build-Prompts sequence

Each feature becomes a dispatch lane prompt. The lane prompt is what an agent can actually consume. This is the bridge between planning and shipping.

The Feature-to-Build-Prompts sequence is the most important prompt in the kit and the one I run most often. It takes a single feature row from Prompt 2's output and produces a complete lane prompt that an agent can pick up and ship from. The full lane structure — verbatim instructions, success criteria, blockers to flag, standing rules — is in the Dispatch Playbook (\$15, paired purchase recommended). This prompt produces lane prompts that already follow that structure.

You are writing a lane prompt for a parallel coding agent. The agent will operate in its own git worktree on a branch named `task/[short-feature-slug]`. Read the feature row below.

Produce a lane prompt with EXACTLY these blocks, in this order:

1. VERBATIM ASK — one paragraph. The original feature description, copied verbatim from the customer where possible. Do not paraphrase.
2. SUCCESS CRITERIA — exactly three observable checks the agent will run after work is complete:
 - a) commit SHA verified on origin/main
 - b) grep proving the change is visible on origin/main
 - c) behavior verification (curl, screenshot, build log, or equivalent depending on the feature)
3. BLOCKERS — name the 3-5 most likely edge cases. For each one, write: "If you hit X, stop and report. Do not improvise."
4. STANDING RULES — link to the standing files. Do not restate the rules. Use this exact line: "Follow CLAUDE.md, AGENTS.md, and the five standing rules in the Dispatch Playbook §5."
5. TOOL TIPS — name the 1-2 local potholes the agent should know about (FUSE-EPERM via Desktop Commander, token paths, etc.).

Hard rules:

- The verbatim ask is the customer's words. Don't translate them.
- Success criteria are commands or facts, not aspirations.
- Blockers are concrete. "If the migration file already exists at the intended timestamp" – not "if anything seems off."

Feature row: \[paste a row from Prompt 2's output\]

The output is a ready-to-spawn lane prompt that fits on one screen. The Dispatch Playbook explains the lane discipline; this prompt assembles the prompt itself.

SECTION 6.

Prompt 4 — Branding extraction

Pull the client's existing brand off their existing site, or — if there isn't one — generate three branded options for them to pick.

Branding extraction is two prompts in one — the **they already have a brand** path and the **they don't yet** path. The kit includes both. The first one takes a live URL and extracts logo, color palette, typography, and copy tone into a structured branding doc plus a Tailwind config + brand-tokens file. The second one generates three branded options based on the spec from Prompt 1, calibrated for the audience.

```
\[BRAND EXTRACTION – when client has a site\]
```

Visit the URL. Produce a structured brand doc with:

- Logo (download URL or extracted SVG if accessible)
- Primary color (hex, named role: primary / accent / etc.)
- Secondary color
- Background color (light + dark variants if both exist)
- Type stack – display, body, mono
- Copy tone – three adjectives ("friendly, direct, slightly technical") + one example phrase from the site

Then produce two files:

1. tailwind.config.ts – extending the default theme with the extracted brand tokens. Named roles, not raw hex.
2. brand-tokens.ts – TypeScript constants for any value the tailwind config can't express (logo URL, copy tone phrases).

Hard rules:

- Only extract values you can verify from the actual rendered site.
- If a value is ambiguous (multiple primary candidates), list all candidates with the page each one came from.
- Don't invent a copy tone. Quote the site.

URL: \[paste live site URL\]

The “don't invent” rule is, again, the part that matters. The default LLM behavior is to produce a “nice-looking” brand doc that might be entirely fabricated. The extraction-from-actual-rendered-site rule keeps the output traceable.

SECTION 7.

Prompt 5 — Migration path

The client has data. The data lives in tools you didn't build. The migration prompt produces an extract → transform → load plan with a reconciliation checklist.

The Migration Path prompt takes the tool list from discovery question five plus a one-paragraph "what to migrate" framing and produces a migration plan in three named stages: extract, transform, load. The reconciliation checklist at the end is what catches the "we migrated 1,247 records but only 1,193 are in the new system" problems before they become Tuesday-evening fires.

You are a data migration planner. Read the tool list and migration scope below.

Produce a migration plan with these stages:

1. EXTRACT — for each source tool, name:
 - Export format (CSV, API, manual)
 - Records to capture (table-by-table list)
 - Sensitive fields requiring handling (PII, payment info, secrets)
2. TRANSFORM — for each source-to-target field mapping:
 - Source field
 - Target field
 - Any normalization required (date formats, currency, etc.)
 - Fields with no clean target (these go to OPEN QUESTIONS)
3. LOAD — name the load tool (Supabase pgbouncer COPY, a one-time migration script, manual paste — pick the cheapest viable option) and the load order (parent tables before children).
4. RECONCILE — three checks:
 - Count matches: SELECT COUNT(*) on source vs target for each table
 - Spot-check: 5 random records compared field-by-field
 - The customer's gut-check question: \[insert from discovery\]
5. WHAT TO LEAVE BEHIND — name the data we're NOT migrating, with the reason (stale, redundant, not worth the effort).

Hard rules:

- If a field cannot be mapped, it goes to OPEN QUESTIONS. Don't invent a target.
- Sensitive fields require an explicit handling line. No "we'll figure it out at load time."

Tool list: \[paste from discovery\
Migration scope: \[paste one paragraph\

The “what to leave behind” stage is the part most operators skip. Skipping it means you migrate every stale record from a 2019 CRM into the shiny new system and inherit ten years of legacy. The explicit “with the reason” forcing function keeps the leave-behind list honest.

SECTION 8.

Prompt 6 — Handoff doc

The build is shipped. The client needs to actually run it. The handoff prompt produces the doc that lets them — without calling you every Tuesday.

The Handoff Doc prompt produces two artifacts: a client-facing PDF (“how to log in, how to add users, how to read the analytics, who to call when X breaks”) and an internal handoff checklist for you (API key swap, billing transition, Vercel/Supabase ownership transfer). The 60-day support window mechanic — what’s covered, what isn’t, how to extend — is calibrated into the doc.

You are writing two handoff artifacts. Read the deployed system description below.

Artifact 1 (client-facing PDF):

1. How to log in (URL, password reset flow)
2. How to perform the 5 most common operations (named explicitly)
3. How to add a team member
4. How to read the analytics dashboard (3 metrics that matter)
5. Who to call when X breaks (escalation tiers + response window)

Hard rules: write at a reading level the customer named in discovery. Don't use developer vocabulary. Use named screenshots where words don't fit.

Artifact 2 (internal handoff checklist):

1. API keys to rotate (named tool + named field)
2. Billing transition (Stripe Connect, Vercel team transfer, Supabase project ownership, domain transfer)
3. Monitoring handover (where alerts go now, who responds)
4. The 60-day support window — what's in, what's out, what triggers a paid extension

System description: \[paste the deployed system overview\]

Customer audience: \[paste from discovery — reading level, role\]

The two-artifact split is the part that earns this prompt its place. One doc for the client, one for you. Mixing the two — the most common failure mode — produces a doc the client can’t read AND a checklist you can’t action.

SECTION 9.

Case study — Patrick, end to end

Fresh Page Home Improvement. A real contractor SaaS, shipped in four days, billed at \$3,500 plus monthly hosting. Every prompt in this kit, in order, with the real inputs and outputs.

Patrick runs Fresh Page Home Improvement — siding, gutters, and roofing in West Tennessee. His pain on the discovery call: every Tuesday morning, he spent 90 minutes flipping between three spreadsheets, a CRM that had never been the right fit, and his texts with crews — trying to figure out which jobs needed quotes that week, which were waiting on insurance approval, and which crews were free. The 90 minutes was Tuesday. The implicit second-90 was Wednesday morning when he had to explain to his office manager what he'd decided. Three hours a week, every week, of work the system should have been doing.

The four-day arc:

DAY 1, AM	Discovery call (45 minutes), nine questions answered. Transcript cleaned to plain text. Prompt 1 ran on transcript + framing — produced the five-section spec.
DAY 1, PM	Prompt 2 ran on the spec — produced the leverage-ranked feature list. Top three: lead intake with SMS notification, quote builder with PDF export, crew load board. Patrick approved the order over text.
DAY 2	Prompt 3 produced lane prompts for each top-three feature. Spawned three dispatch lanes in parallel. All three landed by end of day. Prompt 4 ran on the existing Fresh Page site — extracted the navy + gold color palette, the slab-serif display font, and the copy tone ("direct, slightly Southern, no hype").
DAY 3	Migration prompt ran on the three-spreadsheet history. 1,247 leads extracted, normalized, loaded. Reconciliation found 13 records with missing crew assignments — flagged for Patrick. The leave-behind list cut 312 stale records from 2019-2021.
DAY 4, AM	Handoff doc generated. Client PDF, three pages. Internal checklist, two pages. Walkthrough call with Patrick (30 minutes, recorded). One additional feature added on the call — the contractor-side reminder when an insurance claim hadn't moved in 14 days — built as a single dispatch lane in the afternoon.
DAY 4, PM	Stripe billing wired for monthly hosting (\$249/month). Patrick paid the \$3,500 build invoice via ACH. Production handoff complete. 60-day support window started.

The four days were not eight-hour grinds. Most of the work happened during dispatch lanes running in parallel while I made coffee, replied to emails, and prepped the next prompt. The

actual high-attention work — running discovery, calibrating the prompts to Patrick’s specifics, reviewing lane reports, calling Patrick — was probably ten hours total across the four days.

Patrick has been on the system for nine months now. The reminder feature he added on day four has flagged \$47,000 of stale insurance claims into actionable status. He pays \$249/month for hosting. He has referred two other contractors who are in discovery. The build paid for itself the first quarter. Everything since then has been compounding.

WHAT'S NOT IN THE KIT

The actual brand templates, the contractor-os-template fork, Stripe Connect setup for client billing, the full BOS pricing scripts, the 60-day handoff playbook in depth — those live in the \$997 course (working title: AI Pocket Agency — The Build System) shipping later this year. This kit is the prompts. The course is the full agency model around them.

The Dispatch Playbook is the lane discipline that makes Prompt 3 actually ship. The Dev-Team Document Set is the artifact stack your agents read at session start. The CLAUDE.md Template Library is the on-ramp that gets a new build to “agent is useful” in 30 minutes. Together they’re the kits-side stack. The AI Pocket Agency community at aipocketagency.com — \$47/month, locked for life for the founding 50 — is the live system that evolves the prompts as the agent landscape changes.

The kit is the machine. The community is the maintenance. Both are how you turn a 45-minute call into shipped software next Wednesday.

— Chase